Using Markov chain theory to investigate when to place limit orders in a simple model of a market

Piotr Stanislawski

October 2026

Introduction and Motivation

Most of the textbook examples of Markov chains concern random walks or other relatively abstract stochastic processes. However, Markov chains are introduced as a powerful tool in applied mathematics, and here I will explore an application to financial markets as a specific example of this.

In particular, we will model *limit order books* using a simple Markov chain. This has been explored in texts like *A Stochastic Model for Order Book Dynamics*, though of course in much more advanced scenarios than this one. Hopefully, this model nonetheless provides an example of a reasonable problem to investigate using stochastic processes.

I will construct a much simpler discrete-time model, using the theory of hitting probabilities, hitting times and transition matrices. The aim is to find out when placing a limit order might be better than buying immediately, without needing to simulate the Markov chain.

Some terms in the language of markets are given below. Naturally, these are still somewhat unfamiliar to me, which also contributes to the limited nature of this model.

A *market order* is an instruction to buy or sell an asset immediately at the best available prices. A market buy order will normally begin executing against the lowest available *asks* (sell prices), while a market sell order will begin executing against the highest available *bids* (buy prices).

A *limit order* is an instruction to buy an asset at no more than some specified price, or to sell an asset at no less than some specified price. Unlike a market order, a limit order need not execute immediately, and may never execute at all. The advantage is that it gives the trader control over the price at which they are willing to trade.

Model

Suppose we have a stock trading at the following prices.

The best bid is \$99.99.

The best ask is \$100.01.

Here, we define best bid as the highest price currently offered by any buyer in the market and best ask as the lowest price currently requested by any seller in the market.

Hence, if you want to sell an asset immediately using a market order, you sell at the best bid, and if you want to buy an asset immediately you buy at the best ask.

You want to buy one share; we assume there are two choices.

Choice A is a market order; you buy immediately at \$100.01. Execution is effectively immediate under our assumptions, but you pay the higher price.

Choice B is a limit order; you submit an order to buy at \$99.99. If somebody sells to you, you save \$0.02 relative to buying at the ask. However, your order might never execute, or the stock price could rise while you're waiting. Note also that even if somebody sells at \$99.99, your order might not execute because other orders could be ahead of yours.

We have the following natural questions to ask:

- How does your probability of execution depend on how far back you are in the queue?
- How long should you expect to wait?
- How dependent are these processes on the volatility of the stock?

Most fundamentally, we ask: when do the potential savings justify the risk?

We let $X_n \in \{0, 1, \ldots, K, F, A\}$, where:

- $X_n = q$ means that there are q orders ahead of yours in the queue.
- $X_n = F$ means that your order has been filled.
- $X_n = A$ means that the price has moved unfavourably before your order was filled, and you withdraw it.

In the language of Markov chains, the states F and A are absorbing states.

Modelling this as a discrete-time process, at each time step we may have:

- The queue advances by one position. If you are already first, an incoming sell order instead fills yours.
- An unfavourable price movement occurs, ending the attempt.
- Neither event occurs, so your state is unchanged.

Let's define $d_q = P(X_{n+1} = q - 1 \mid X_n = q)$ for $q \geq 1$, with $d_0 = P(X_{n+1} = F \mid X_n = 0)$, and $a_q = P(X_{n+1} = A \mid X_n = q)$. Thus, d_q is the probability that the queue advances, or that our order executes if $q = 0$, while a_q is the probability of an unfavourable price move. We require $d_q + a_q \leq 1$ always.

The probability of staying in a particular transient state is $1 - d_q - a_q$.

An example of the transition matrix for such a Markov chain with $K = 2$, using the state order $(0, 1, 2, F, A)$, is given below.

$$P = \begin{pmatrix} 1 - d_0 - a_0 & 0 & 0 & d_0 & a_0 \\ d_1 & 1 - d_1 - a_1 & 0 & 0 & a_1 \\ 0 & d_2 & 1 - d_2 - a_2 & 0 & a_2 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

The final two rows make the absorbing nature of F and A explicit. Each row sums to 1, so P is a stochastic matrix.

We have now described our most fundamental model.

Assumptions

We have made quite a few assumptions to keep the model simple and easy to understand. After all, the main motivation is to provide an example of how Markov chains may be used in financial markets rather than a necessarily accurate model.

- We consider one buy order for one share at the best bid. We assume all orders ahead of ours have the same size, and that each queue advancement removes exactly one of them. The queue follows price–time priority, so new orders do not jump ahead of ours.
- The probabilities d_q and a_q depend only on the current queue position. They do not depend on time or how we reached that position. This is what makes our process a homogeneous Markov chain.
- We have not used actual market data. The probabilities are arbitrary examples, and the time steps do not represent any particular amount of time.

Probability of Execution

We have the probability of execution

$$h_q = P_q(T_F < T_A),$$

where the notation P_q refers to a probability conditioned on starting at state q, and T_F, T_A are the hitting times of the two absorbing states.

This is the hitting probability of the set $\{F\}$, and so h_q is an absorption probability.

We have the following theorem for hitting probabilities, which may be proven by conditioning on the first transition (the partition theorem) and applying the Markov property.

Theorem

For the hitting probability h_i^B of a subset B from a starting state i, we have

$$h_i^B = \sum_j p_{i,j} h_j^B \qquad \text{for } i \notin B,$$

and $h_i^B = 1$ for $i \in B$.

Applying this to our Markov chain, we have

$$h_q = \sum_j p_{q,j} h_j, \qquad h_F = 1, \qquad h_A = 0.$$

The latter boundary condition holds because, once the chain reaches the absorbing state A, the order can never be filled.

We will now provide an explicit formula for the probabilities of execution.

Starting with $q = 0$, we have

$$h_0 = p_{0,F} h_F + p_{0,0} h_0 + p_{0,A} h_A$$

We may ignore all other terms as then $p_{0,j} = 0$. Hence,

$$h_0 = d_0 + (1 - d_0 - a_0) h_0,$$

and

$$h_0 = \frac{d_0}{d_0 + a_0}$$

Now take $q \geq 1$. We have similarly

$$h_q = p_{q,q-1}h_{q-1} + p_{q,q}h_q + p_{q,A}h_A.$$

Then

$$h_q = \frac{d_q}{d_q + a_q}h_{q-1}$$

We have a recurrence, and this results in

$$h_q = \prod_{k=0}^{q} \frac{d_k}{d_k + a_k}$$

Thus, the execution probability is a product of the probabilities of successfully advancing through each queue position before an unfavourable movement occurs.

We also have $h_{q+1} \leq h_q$, since the formula for h_{q+1} is just that for h_q multiplied by a number between 0 and 1. Hence, the further back we are in the queue, the smaller our probability of execution.

A generalisation

The product formula is very useful in this simple model, but it only applies to this specific model. In situations where, for example, we have states which encode the calmness or volatility of the market, or any number of other more complicated models, we can also calculate the hitting probabilities using matrices.

Put the transient states first and the absorbing states F, A last. Then the transition matrix has the form

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where Q contains the probabilities of moving between transient states, R contains the probabilities of moving from a transient state to an absorbing state, and I is the 2×2 identity matrix.

Let $h = (h_0, \ldots, h_K)'$ and let r be the first column of R, which gives the probabilities of being filled in one step. Our equations for the hitting probabilities can now be written as

$$h = Qh + r.$$

So

$$(I - Q)h = r$$

and therefore

$$h = (I - Q)^{-1}r$$

In Python, we will solve the system $(I - Q)h = r$ directly, rather than actually calculate the inverse.

We are now able, by the above method, to answer the first of our questions.

Expected Waiting Times

We also want to know how long we should expect to wait. Let

$$T = \min\{T_F, T_A\}, \qquad t_q = E_q(T).$$

This is the time until the order is either filled or cancelled. Notice that it is not the expected time until a fill, conditional on the order actually being filled.

As before, we condition on the first transition. We use one time step, and then, if we are still in a transient state j, we expect to wait another t_j steps. Thus,

$$t_q = 1 + \sum_{j=0}^{K} p_{q,j} t_j.$$

Writing $t = (t_0, \ldots, t_K)'$, we have

$$t = \mathbf{1} + Qt,$$

so

$$t = (I - Q)^{-1}\mathbf{1}$$

where $\mathbf{1}$ is the column vector of ones.

For our particular model, we could instead solve the recurrences

$$t_0 = \frac{1}{d_0 + a_0}, \qquad t_q = \frac{1 + d_q t_{q-1}}{d_q + a_q}$$

However, the matrix method lets us calculate all the expected times in the same way as the execution probabilities.

There is also a useful interpretation of the matrix $(I - Q)^{-1}$ which can be used to obtain the same result. Since

$$(I - Q)^{-1} = I + Q + Q^2 + \cdots$$

Now the Chapman-Kolmogorov equations state P^n give the n-step transition probabilities, and P^n has upper-left entry (in the simplified form) Q^n. So the (i, j)th entry of $(I - Q)^{-1}$ is the expected number of visits to state j before absorption when starting at i.

This is called the fundamental matrix of the absorbing chain. Summing its rows gives the expected times t_i.

A Numerical Example

Suppose there are at most three orders ahead of ours and we choose the following hypothetical transition probabilities:

$$d_0 = 0.40, \qquad d_1 = 0.30, \qquad d_2 = 0.25, \qquad d_3 = 0.20,$$

with $a_q = 0.10$ for all q.

For example, at position 0 there is a probability 0.40 of immediate execution, a probability 0.10 of an unfavourable move, and a probability 0.50 of remaining at the front of the queue in the next step.

The transient transition matrix and the two absorption columns are

$$Q = \begin{pmatrix} 0.50 & 0 & 0 & 0 \\ 0.30 & 0.60 & 0 & 0 \\ 0 & 0.25 & 0.65 & 0 \\ 0 & 0 & 0.20 & 0.70 \end{pmatrix}, \qquad R = \begin{pmatrix} 0.40 & 0.10 \\ 0 & 0.10 \\ 0 & 0.10 \\ 0 & 0.10 \end{pmatrix}.$$

The execution probabilities follow from our explicit formula:

$$\begin{aligned} h_0 &= \frac{0.40}{0.50} = 0.80, \\ h_1 &= \frac{0.30}{0.40} h_0 = 0.60, \\ h_2 &= \frac{0.25}{0.35} h_1 = \frac{3}{7} \approx 0.4286, \\ h_3 &= \frac{0.20}{0.30} h_2 = \frac{2}{7} \approx 0.2857. \end{aligned}$$

We obtain the expected waiting times from $(I - Q)t = \mathbf{1}$:

$$t_0 = 2, \qquad t_1 = 4, \qquad t_2 = \frac{40}{7} \approx 5.71, \qquad t_3 = \frac{50}{7} \approx 7.14.$$

Notice that starting further back reduces the probability of execution and increases the average time until either outcome. This makes sense intuitively.

Expected Execution Costs

We now return to our main question: when is it worth waiting for a limit order rather than just buying immediately?

Let b be the best bid and s the best ask.

Suppose an unfavourable movement raises the ask by $\Delta > 0$. If this happens, we cancel our limit order and buy at the new ask $s + \Delta$.

We will also allow a small cost c for each step spent waiting, representing the disadvantage of having to wait.

A market order costs $C := s$.

For the limit order, we pay b if it is filled, and $s + \Delta$ if it is not. We also pay the waiting cost for however long the attempt lasts. Hence,

$$C(q) = bh_q + (s + \Delta)(1 - h_q) + ct_q$$

for $C(q)$, the expected cost of obtaining a share after initially placing a limit order at the qth position in the queue.

It is better to use the limit order if $C(q) < C$. Rearranging this gives

$$h_q(s - b + \Delta) > \Delta + ct_q$$

This is essentially the condition we wanted. Whether waiting is worthwhile depends on both how likely our order is to be filled and how long the attempt is expected to take.

Like h_q and t_q, we do not know Δ exactly before placing an order, and it is likely in fact a random variable. However, we may be able to obtain an estimate from prior market data.

An application

We will now apply our results to decide whether a limit order or a market order is preferable.

Recall that for our example, we had

$$d_0 = 0.40, \qquad d_1 = 0.30, \qquad d_2 = 0.25, \qquad d_3 = 0.20$$

and $a_q = 0.10$ for all q.

Using the hitting probability and hitting time equations for our Markov chain, we obtained

$$h = \begin{pmatrix} 0.80 \\ 0.60 \\ 3/7 \\ 2/7 \end{pmatrix}, \qquad t = \begin{pmatrix} 2 \\ 4 \\ 40/7 \\ 50/7 \end{pmatrix}.$$

These are the quantities we need to make our decision.

We take the best bid and ask to be

$$b = 99.99, \qquad s = 100.01,$$

and assume a waiting cost of $c = 0.001$ per step.

Let $\Delta = 0.01$, so that if our limit order does not execute, we cancel it and buy at the new ask of \$100.02.

Recall that the expected cost of a limit order is

$$C(q) = bh_q + (s + \Delta)(1 - h_q) + ct_q,$$

while the cost of a market order is $C = s$.

Substituting our values of h_q and t_q, we obtain

$$\begin{aligned} C &= 100.0100 \\ C(0) &= 99.9980 \\ C(1) &= 100.0060 \\ C(2) &\approx 100.0129 \\ C(3) &\approx 100.0186 \end{aligned}$$

Since $C = 100.01$, the limit order is preferable when $q = 0$ or $q = 1$, but not when $q = 2$ or $q = 3$.

Thus, under these assumptions, we should place a limit order if there are fewer than two orders ahead of ours, and otherwise buy immediately.

We can also obtain a more general result. Recall that a limit order is preferable if

$$h_q(s + \Delta - b) > \Delta + ct_q$$

Rearranging, we obtain

$$\Delta < \frac{h_q(s - b) - ct_q}{1 - h_q}$$

assuming $h_q < 1$.

The right hand side gives the maximum adverse price movement for which a limit order is preferable. Using our values of h_q and t_q, we obtain

$$\begin{aligned} q = 0: &\quad \Delta < 0.07 \\ q = 1: &\quad \Delta < 0.02 \\ q = 2: &\quad \Delta < 0.005 \\ q = 3: &\quad \Delta < -0.002 \end{aligned}$$

Hence, whenever $0.005 < \Delta < 0.02$, we should place a limit order if $q < 2$ and a market order otherwise. This includes our choice of $\Delta = 0.01$.

Notice also that for $q = 3$, the threshold is negative. Since we are assuming $\Delta > 0$, placing a limit order at this position is never preferable under our chosen probabilities and waiting cost.

If we replace the fixed quantity Δ with a random variable D, it follows that the criterion

$$E(D) < \frac{h_q(s-b) - ct_q}{1 - h_q}$$

holds also.

Sensitivity Analysis

So far, we have only looked at one set of probabilities. We can use Python to see what happens if we change them.

For simplicity, keep the queue-advancement probabilities d_q fixed and take the same adverse-movement probability $a_q = a$ at every position.

We will compare $a = 0.05$, 0.10 and 0.20.

From

$$h_q = \prod_{k=0}^{q} \frac{d_k}{d_k + a},$$

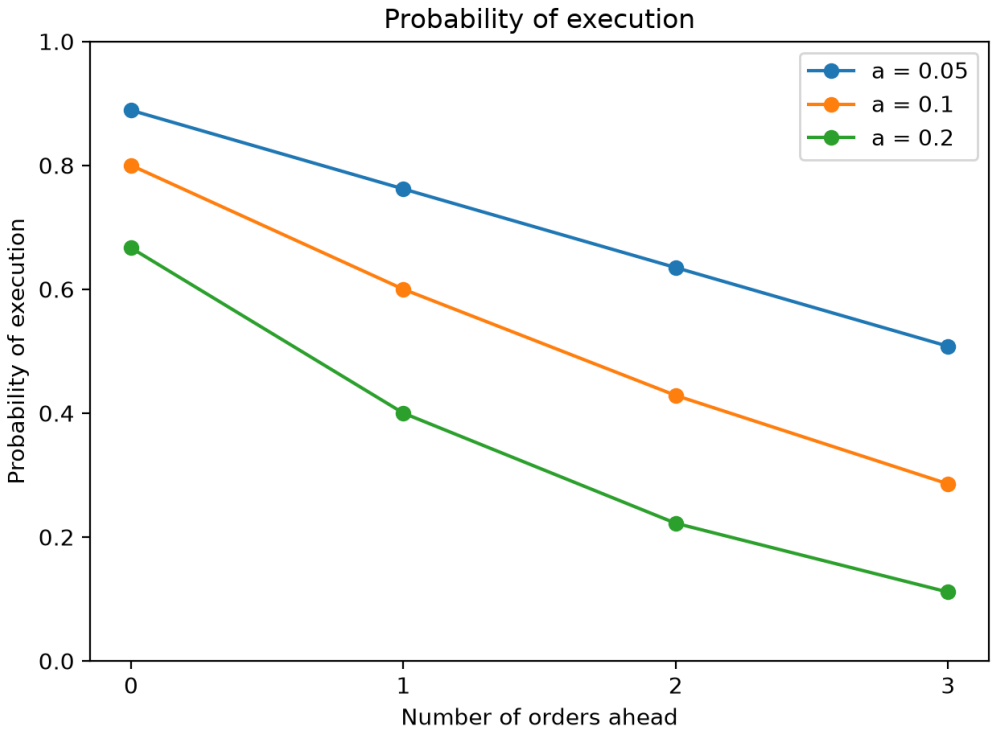
we can see that increasing a decreases the probability of execution.

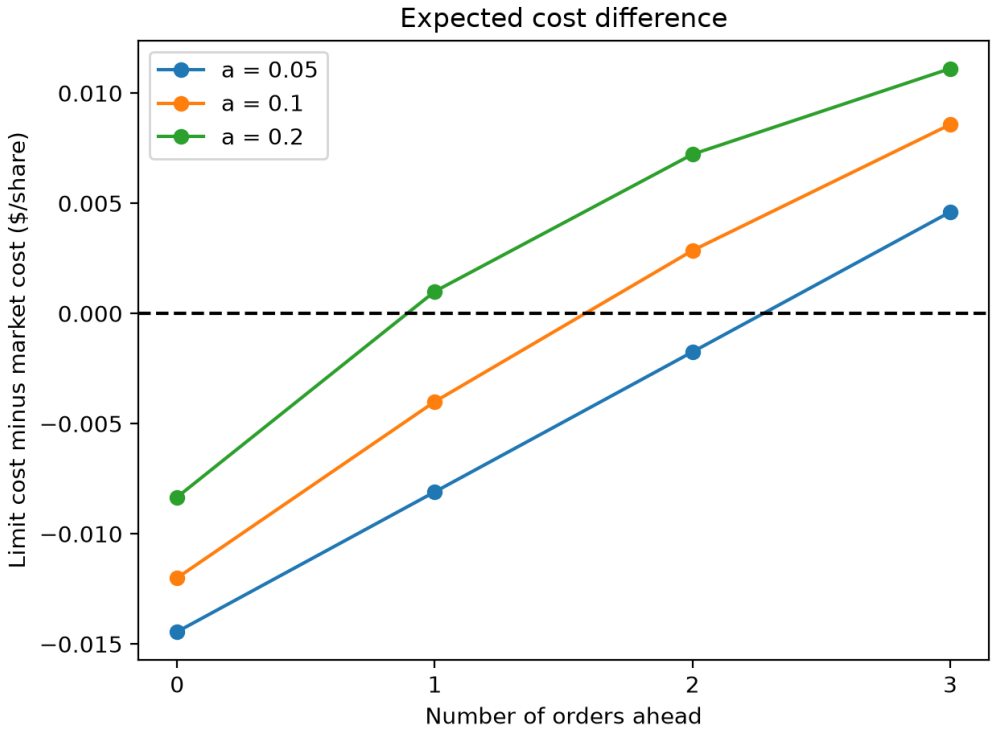
This is what we would expect: the price is more likely to move against us before we reach the front of the queue.

We will plot the fill probabilities and the difference

$$C(q) - C$$

against queue position for the three values of a.





In the second plot, values below 0 mean that the limit order is cheaper under our assumptions.

There is an interesting tradeoff here: a larger a makes execution less likely, but it may also mean the attempt ends sooner.

Hence, it is worth looking at both the hitting probabilities and the expected times, rather than just one of them.

Implementation in Python

We can implement the calculations using NumPy.

The main task is to construct Q and R and solve the two systems

$$(I - Q)h = r, \qquad (I - Q)t = \mathbf{1}$$

We will also calculate the costs and make the two plots described above using Matplotlib.

Below is the Python code. As a check, it compares the hitting probabilities found using matrices with our explicit product formula, and checks that the two absorption probabilities add to 1.

```
import numpy as np
import matplotlib.pyplot as plt


# This function works out the probabilities, waiting times and costs
# d is the list of probabilities of moving forward in the queue
# a is the probability of an adverse price movement
# delta is how much the ask goes up if that happens

def calculate(d, a, delta):
    n = len(d)  # number of queue positions (0, 1, ..., n-1)

    # Q is for moving between queue positions
    # R is for going to one of the absorbing states F or A
    Q = np.zeros((n, n))
    R = np.zeros((n, 2))

    # Fill in the matrices one row at a time
    for q in range(n):
        Q[q, q] = 1 - d[q] - a  # nothing changes
        R[q, 1] = a             # price moves against us

        if q == 0:
            R[q, 0] = d[q]      # our order gets filled
        else:
            Q[q, q - 1] = d[q]  # one fewer order ahead of us

    # Work out I-Q, which appears in both of our equations
    M = np.eye(n) - Q

    # First solve (I-Q)h=r to get the fill probabilities
    h = np.linalg.solve(M, R[:, 0])

    # Then solve (I-Q)t=1 to get the expected times
    t = np.linalg.solve(M, np.ones(n))

    # The other absorbing outcome has probability 1-h
    # Work out the expected cost of starting with a limit order
    cost = 99.99 * h + (100.01 + delta) * (1 - h) + 0.001 * t
```

```
    return h, t, cost


# Now there can be as many as 3 orders ahead of ours
# The first three values are the ones from the write-up
# We have chosen d_3=0.20 for the extra position

d = [0.40, 0.30, 0.25, 0.20]
a = 0.10
delta = 0.01

h, t, costs = calculate(d, a, delta)

# Print the answers for the main example
print("Market order cost: $100.0100")
print("Results for a fixed price increase of $0.01:")

for q in range(len(d)):
    print("q =", q)
    print("  Probability of execution:", round(h[q], 4))
    print("  Expected time:", round(t[q], 4))
    print("  Expected cost: $", format(costs[q], ".4f"), sep="")

    if costs[q] < 100.01:
        print("  Limit order is cheaper")
    else:
        print("  Market order is cheaper")


# The break-even increase in the ask, from our formula
print("\nBreak-even price increases:")

for q in range(len(d)):
    value = (h[q] * 0.02 - 0.001 * t[q]) / (1 - h[q])
    print("q =", q, ": delta <", round(value, 4))


# We can also let the price increase be random
# Here it is $0.01 with probability 3/4 and $0.03 with probability 1/4
# Because we only want the expected cost, we just use E(D)

mean_delta = 0.75 * 0.01 + 0.25 * 0.03
h2, t2, random_costs = calculate(d, a, mean_delta)

print("\nRandom price increase, with mean", round(mean_delta, 4))

for q in range(len(d)):
    print("q = ", q, " : expected cost = $",
          format(random_costs[q], ".4f"), sep="")
```

```
# Check that our answers agree with the product formula we derived
print("\nProbabilities from the product formula:")

answer = 1

for q in range(len(d)):
    answer = answer * d[q] / (d[q] + a)
    print("q =", q, ":", round(answer, 4))


# First graph: how the fill probability changes with a
positions = range(len(d))

for a in [0.05, 0.10, 0.20]:
    h, t, costs = calculate(d, a, delta)
    plt.plot(positions, h, marker="o", label="a = " + str(a))

plt.xlabel("Number of orders ahead")
plt.ylabel("Probability of execution")
plt.title("Probability of execution")
plt.xticks(list(positions))
plt.ylim(0, 1)
plt.legend()
plt.tight_layout()
plt.savefig("fill_probabilities.png", dpi=200)
plt.close()


# Second graph: compare the costs of limit and market orders
# Below zero means the limit order is cheaper

for a in [0.05, 0.10, 0.20]:
    h, t, costs = calculate(d, a, delta)
    plt.plot(positions, costs - 100.01, marker="o",
             label="a = " + str(a))

plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Number of orders ahead")
plt.ylabel("Limit cost minus market cost ($/share)")
plt.title("Expected cost difference")
plt.xticks(list(positions))
plt.legend()
plt.tight_layout()
plt.savefig("execution_costs.png", dpi=200)
plt.close()
```

This is enough to get the calculations and two plots without simulating any random paths.

This lack of any actual simulation justifies one of the goals which we aimed to show, namely how Markov chain theory can be used to investigate a question about trading. Of course, the simplicity of the model compared to a real market must also be considered.

Sources and References

The results in Markov chain theory which were applied are some of those in *Grimmett and Welsh's Probability: An Introduction*.

The idea for the project - that is, to use Markov chain theory to model an order book - came from the much more advanced examples in *Cont, Stoikov, and Talreja's A Stochastic Model for Order Book Dynamics*, which introduce far more applicable (and harder to understand!) models in continuous time.

AI Disclosure: An LLM was used to provide specific numerical examples which would lead to interesting results and proof-read this document for errors and issues in LaTeX formatting. It also helped with debugging code when there were some more obscure errors. Finally, an LLM suggested the addition of the additional interpretation of the matrix $(I - Q)^{-1}$ in the section on expected waiting times and the graphical outputs of the program to this document.

Any errors made are my own.